

FrontRow: Event Ticketing

The scenario

Build **FrontRow**, a seat-level ticketing app for events (concerts, matches, theatre). Each event has a **seat map**; customers pick specific seats and buy them. The whole difficulty is what happens when an event is popular and **many people try to buy the same seats at the same time**: a seat must never be sold twice, yet a buyer needs the seats reserved for a few minutes while they check out, and if they walk away, those seats must free up again automatically. This is mostly a **correctness-under-concurrency** problem, which is exactly what we want to see you reason about. The AI is only one small convenience feature.

What to build

A complete app in four clearly-separated parts. The frontend only ever talks to the backend.

- **Frontend** (React / Next.js): login, browse events, a seat-map view showing available / held / sold seats, a hold-and-checkout flow, and the buyer's tickets.
- **Backend API** (Node.js or Python FastAPI, using an ORM): auth, events & seat maps, and the **hold / purchase / expiry logic** (the heart of the exercise), plus the one endpoint that calls the LLM engine.
- **LLM engine** (Python FastAPI): the only part that talks to the model. It turns a plain-English seat request into a structured preference in **one call**.
- **PostgreSQL** stores users, events, seats and their state, holds, and orders.

Core application features

- Register / log in. Browse events; open an event to see its seat map with each seat's status and price.
- Select one or more seats and place a **temporary hold** (e.g. a 5-minute reservation) so no one else can take them while you decide.
- Complete checkout to convert a hold into a confirmed **order** with tickets; or let the hold **expire**; expired holds release their seats back to available automatically.
- See your orders / tickets. (Payment can be a simple mock, no real payment processing needed.)

The hard part: this is what we're really testing

Your design must guarantee, and you must be able to explain how:

- **No double-selling.** If two people try to hold or buy the same seat at the same instant, exactly one succeeds and the other gets a clear "seat no longer available", even under concurrent requests.
- **Time-limited holds.** A hold reserves specific seats for a fixed window; during it, those seats are unavailable to everyone else.
- **Automatic expiry.** If checkout doesn't complete in time, the hold expires and the seats become available again, whether via a background job, lazy expiry on read, or however you choose.

Think about how you model seat state and how you enforce these rules at the **database level** (transactions, row locking, unique constraints, or optimistic concurrency, your call), not just in application code. In your notes, briefly describe your approach and how you'd convince yourself it's correct.

The one AI feature (just connect an LLM, no RAG, no vector DB)

"Find me seats." The buyer types a request like **"2 seats together, cheapest, near the front."** The backend sends it to the LLM engine, which makes a **single, direct call** and returns a structured preference as validated JSON (`{quantity, adjacency, max_price, preferred_section}`). Your own code then uses that preference to pick matching seats from the real, currently-available seat map; the model only interprets the request, it never decides availability. Validate the JSON, and make sure everything still works if the model is slow or down (the buyer can always pick seats manually). Any provider is fine (OpenAI, Claude, Gemini, or Ollama), behind a swappable interface with the API key in environment variables.

What we're looking for

Above all, a correct concurrency design you can defend: clean seat-state modelling and the right database-level guarantees against double-selling. Then the usual: clean separation between the parts, sensible API design, graceful failure handling, and readable code. For the AI feature: validated output and a clean boundary (interpretation in the model, decisions in your code). A correct, well-reasoned core beats a long feature list. Using AI coding tools is fine; you should be able to explain every line, especially the concurrency.

Deliverables

A Git repo (frontend, backend, LLM engine, DB migrations), a short README to configure and run it, a `.env.example`, and seed data (an event with a small seat map) so we can try it right away. Strongly encouraged: a short note (or a test) showing how your design prevents two buyers getting the same seat. docker-compose optional.